Accelerated Radiative Diffusion Problems

Original article: Synthetic Acceleration for Radiative Diffusion Calculations [1]

Kyle Hansen

NE 795 Computational Project
12 December 2025

1. Problem

The equations for radiative diffusion are

$$\frac{1}{c}\frac{\partial I}{\partial t} - \nabla \cdot D(\nu)\nabla I(\nu) + \rho\kappa(E)I(E) = \rho\kappa(\nu)\beta(\nu, T) \tag{1a}$$

$$\rho C_v \frac{\partial T}{\partial t} = \nabla \cdot K\nabla T - \nabla \mathbb{P}u + \rho \int_0^\infty \kappa(E') \left[I(E') - \beta(E', T)\right] dE' + Q, \tag{1b}$$

where Equation 1a is the radiative diffusion equation, where c is the speed of light, I is scalar intensity $I_\nu(x) = \int_{4\pi} \mathcal{I}_\nu(x, \Omega) d\Omega = cE$, D is the radiative diffusion constant $(3\rho\kappa)^{-1}$, κ is specific opacity [cm^2/g], and β is the angle-integrated Planck function, $\beta_\nu(T) = \int_{4\pi} B_\nu(T) d\Omega$.

Equation 1b is the heat diffusion equation with radiation and hydrodynamic effects, where ρ is material density, C_v is material heat capacity, K is thermal conductivity, $\mathbb{P}$ is the fluid pressure tensor, u is fluid velocity, and Q is heat generation per unit mass.

The energy domain can be discretized using N_g energy groups $G_i = [\nu_i, \nu_{i+1}]$ such that $\nu_0 = 0$ and $\nu_{N_g+1} = \infty$. Using this discretization, and using $\varkappa = \rho\kappa$, the diffusion equations become

$$\frac{1}{c}\frac{\partial I_k}{\partial t} - \nabla \cdot D_k \nabla I_k + \varkappa_k I_k = \varkappa_k \beta_k(T) \tag{2a}$$

$$\rho C_v \frac{\partial T}{\partial t} = \nabla \cdot K\nabla T - \nabla \mathbb{P}u + \sum_{g=1}^{N_g} \varkappa_g \left[I_g - \beta_g(T)\right]' + Q, \tag{2b}$$

where

$$I_k = \int_{G_k} I(\nu) d\nu \tag{3a}$$

$$\beta_k = \int_{G_k} \beta(\nu, T) d\nu \tag{3b}$$

$$D_k \nabla I_k = \int_{G_k} D(\nu) \nabla I(\nu) d\nu \tag{3c}$$

$$\varkappa_k = \frac{\int_{G_k} \left[\beta(\nu) - I(\nu)\right] \varkappa(\nu) d\nu}{\int_{G_k} \left[\beta(\nu) - I(\nu)\right] d\nu}. \tag{3d}$$

Discretizing in time using Backward Euler (time index n), with lagged coefficients, Equation 1 becomes:

$$\frac{1}{c}\frac{I^{n+1} - I^n}{\Delta t^n} - \nabla \cdot D_k^n \nabla I_k^{n+1} + \varkappa_k^n I_k^{n+1} = \varkappa_k^n \beta_k \left(T^{n+1}\right); \qquad k = 1 \dots N_g \tag{4a}$$

$$\rho C_v^n \frac{T^{n+1} - T^n}{\Delta t^n} = \nabla \cdot K^n \nabla T^{n+1} - \nabla \mathbb{P} u + \sum_{g=1}^{N_g} \varkappa_g^n \left[I_g^{n+1} - \beta_g(T^{n+1})\right] + Q^{n+1}. \tag{4b}$$

The time-advanced Planck function β is approximated using the first-order Taylor expansion $\beta(T^{n+1}) \approx \beta(T^n) + \frac{\partial \beta}{\partial T}(T^{n+1} - T^n)$, which leads to the system for T^{n+1} and I^{n+1},

$$\frac{1}{c}\frac{I^{n+1} - I^n}{\Delta t^n} - \nabla \cdot D_k^n \nabla I_k^{n+1} + \varkappa_k^n I_k^{n+1} = \varkappa_k^n \left(\beta(T^n) + \frac{\partial \beta}{\partial T}\Delta T^{n+1}\right); \qquad k = 1 \dots N_g \tag{5a}$$

$$\rho C_v^n \frac{\Delta T^{n+1}}{\Delta t^n} = \nabla \cdot K^n \nabla T^{n+1} - \nabla \mathbb{P} u + \sum_{g=1}^{N_g} \varkappa_g^n \left[I_g^{n+1} - \beta(T^n) - \frac{\partial \beta}{\partial T}\Delta T^{n+1}\right] + Q^{n+1}. \tag{5b}$$

Finally, the temperature operator is split into the hydrodynamic and radiative contributions,

$$\rho C_v^n \frac{\Delta T^{n+1}}{\Delta t^n} = \nabla \cdot K^n \nabla T^{n+1} - \nabla \mathbb{P} u + Q^{n+1} \tag{6a}$$

$$\frac{1}{c}\frac{I^{n+1} - I^n}{\Delta t^n} - \nabla \cdot D_k^n \nabla I_k^{n+1} + \varkappa_k^n I_k^{n+1} = \varkappa_k^n \left(\beta(T^n) + \frac{\partial \beta}{\partial T}\Delta T^{n+1}\right); \qquad k = 1 \dots N_g \tag{6b}$$

$$\rho C_v^n \frac{\Delta T^{n+1/2}}{\Delta t^n} = \sum_{g=1}^{N_g} \varkappa_g^n \left[I_g^{n+1} - \beta(T^n) - \frac{\partial \beta}{\partial T}\Delta T^{n+1/2}\right], \tag{6c}$$

where $T^{n+1} = T^n + \Delta T^{n+1/2} + \Delta T^{n+1}$. The indices $n+1/2$ and $n+1$ in this case refer to two separate contributions in the same time step, not temperature change at an intermediate time step.

Equation 6a can be immediately solved for temperature change,

$$\Delta T^{n+1/2} = \frac{\sum_{g=1}^{N_g} \varkappa_g \left[I_g^{n+1} - \beta_g^n \right] + Q}{\frac{\rho C_v^n}{\delta t^n} + \sum_{g=1}^{N_g} \varkappa_g \frac{\partial \beta}{\partial T}}, \tag{7}$$

which is then substituted into Equation 6b to give the radiative diffusion with implicit ΔT:

$$\frac{1}{c\Delta t^n} \left(I^{n+1} - I^n \right) - \nabla \cdot D_k^n \nabla I_k^{n+1} + \varkappa_k^n I_k^{n+1} = \eta \chi_k \sum_{g=1}^{N_g} \varkappa_g I_g^{n+1} + q_k \tag{8a}$$

where

$$\eta = \left[\sum_{g=1}^{N_g} \varkappa_g \frac{\partial \beta_g}{\partial T} \right] \cdot \left[\frac{\rho C_v}{\Delta t^n} + \sum_{g=1}^{N_g} \varkappa_g \frac{\partial \beta_g}{\partial T} \right]^{-1} \tag{8b}$$

$$\chi_k = \left[\varkappa_k \frac{\partial \beta_k}{\partial T} \right] \cdot \left[\sum_{g=1}^{N_g} \varkappa_g \frac{\partial \beta_g}{\partial T} \right]^{-1} \tag{8c}$$

$$q_k = \varkappa_k \beta_k + \eta \chi_k \left[Q - \sum_{g=1}^{N_g} \varkappa_g \frac{\partial \beta_g}{\partial T} \right]. \tag{8d}$$

Equation 8a is solved for intensity, then temperature change is recovered afterward. This can be rearranged to further resemble a steady-state neutron diffusion equation,

$$-\nabla \cdot D_k \nabla I_k^{n+1} + \left(\sigma_a + \sigma_{f,k} \right) I_k^{n+1} = \eta \chi_k \sum_{g=1}^{N_g} \sigma_{f,g} I_g^{n+1} + S_k \tag{9}$$

where

$$\sigma_a = \frac{1}{c\Delta t^n} \tag{10a}$$

$$\sigma_{f,k} = \varkappa_k \tag{10b}$$

$$S_k = q_k + \frac{I_k^n}{c\Delta t^n}. \tag{10c}$$

Here $\sigma_{f,k} = \varkappa_k$ acts as a fission cross section, where photons are re-emitted with the "fission" spectrum χ_k, with multiplicity η, and capture cross section σ_a. The physical interpretation is that photons may be lost to material absorption or streaming in time (from $\frac{d}{dt}$). Those photons absorbed in the material are either re-emitted in the same time step, or contribute to temperature gain in the material.

The intensity equation is solved using power iteration on the fission source. Starting from initial iterate $I_k^{(0)}$, the $l+1$ iterate for intensity at the $n+1$ time step is found by solving the linear system

$$-\nabla \cdot D_k \nabla I_k^{l+1} + (\sigma_a + \sigma_{f,k}) I_k^{l+1} = \eta \chi_k \sum_{g=1}^{N_g} \sigma_{f,g} I_g^l + S_k. \tag{11}$$

The full iterative algorithm is displayed in Algorithm 1.

Algorithm 1 Unaccelerated Iteration Algorithm

while $t^n \leq t^{end}$ **do**
  $n \leftarrow n + 1$
  Compute (lagged) temperature-dependent coefficients using $T|_{t=t^{n-1}}$
  while $\left\| I^{(l)} - I^{(l-1)} \right\| > \epsilon \left\| I^{(l-1)} \right\|$ **do**
    $l \leftarrow l + 1$
    $I_k^{(l+1/2)} \leftarrow$ Source iteration on $\eta \chi_k \sum_{g=1}^{N_g} \sigma_{f,g} I_g^{n+1}$ (11)
  end while
  $I_k^n \leftarrow I_k^{(l+1)}$
  Recover $\Delta T^{n+1/2}$ from I_k^n (7)
  if $K \neq 0 \wedge u \neq 0$ **then**
    Solve thermal diffusion equation for ΔT^{n+1}
  end if
  $T^{n+1} \leftarrow T^n + \Delta T^{n+1/2} + \Delta T^{n+1}$
end while

1.1 Synthetic Acceleration Method

The equation for exact multigroup error of the lth iterate (Equation 11) is

$$-\nabla \cdot D_k \nabla \epsilon_k^l + (\sigma_a + \sigma_{f,k}) \epsilon_k^l = \eta \chi_k \sum_{g=1}^{N_g} \sigma_{f,g} \left[\epsilon_g^l + I_g^l - I_g^{l-1} \right]. \tag{12}$$

At equilibrium, this becomes

$$(\sigma_a + \sigma_{f,k}) \epsilon_k^l = \eta \chi_k \sum_{g=1}^{N_G} \sigma_{f,g} \epsilon_g^l + \eta \chi_k \sum_{g=1}^{N_G} \sigma_{f,g} \left(I_g^l - I_g^{l-1} \right) \tag{13}$$

and the group spectrum of ϵ is

$$\frac{\epsilon_k^l}{\mathcal{E}^l} = \frac{\langle \sigma_t \rangle \chi_k}{\sigma_{t,k}} \tag{14}$$

where

$$\mathcal{E}^l = \sum_{g=1}^{N_g} \epsilon_g^l \tag{15a}$$

$$\langle \sigma_t \rangle = \left[\sum_{g=1}^{N_g} \frac{\chi_g}{\sigma_{t,g}} \right]^{-1}. \tag{15b}$$

Then, all groups of Equation 12 are summed, using this spectrum, to get

$$-\nabla \cdot \left[\langle D \rangle \nabla \mathcal{E}^l + \langle D' \rangle \mathcal{E}^l \right] + \mathcal{E}^l \left[\sigma_a + \langle \sigma_f \rangle (1 - \eta) \right] = \eta r^l \tag{16}$$

where

$$\langle D \rangle = \sum_{k=1}^{N_g} D_k \frac{\langle \sigma_t \rangle \chi_k}{\sigma_{t,k}} \tag{17a}$$

$$\langle D' \rangle = \sum_{k=1}^{N_g} D_k \nabla \frac{\langle \sigma_t \rangle \chi_k}{\sigma_{t,k}} \tag{17b}$$

$$\langle \sigma_f \rangle = \langle \sigma_t \rangle \sum_{g=1}^{N_g} \frac{\chi_g \sigma_{f,g}}{\sigma_{t,g}} \tag{17c}$$

$$r^{l+1/2} = \sum_{g=1}^{N_g} \sigma f, g \left(I_g^{l+1/2} - I_g^l \right). \tag{17d}$$

Then the spectrum from Equation 14 can be used to calculate the groupwise correction terms from the grey correction in Equation 16. The accelerated iterative scheme becomes

$$-\nabla \cdot D_k \nabla I_k^{l+1} + \left(\sigma_a + \sigma_{f,k} \right) I_k^{l+1} = \eta \chi_k \sum_{g=1}^{N_g} \sigma_{f,g} I_g^l + S_k \tag{18a}$$

$$-\nabla \cdot \left[\langle D \rangle \nabla \mathcal{E}^{l+1/2} + \langle D' \rangle \mathcal{E}^{l+1/2} \right] + \mathcal{E}^{l+1/2} \left[\sigma_a + \langle \sigma_f \rangle (1 - \eta) \right] = \eta r^{l+1/2} \tag{18b}$$

$$I^{l+1} = I_k^{l+1} + \mathcal{E}^{l+1/2} \frac{\langle \sigma_t \rangle \chi_k}{\sigma_{t,k}} \tag{18c}$$

The accelerated iterative algorithm is displayed in Algorithm 2

Algorithm 2 Accelerated Iteration Algorithm

while $t^n \leq t^{end}$ **do**
  $n \leftarrow n+1$
  Compute (lagged) temperature-dependent coefficients using $T|_{t=t^{n-1}}$
  while $\left\|I^{(l)} - I^{(l-1)}\right\| > \epsilon \left\|I^{(l-1)}\right\|$ **do**
    $l \leftarrow l+1$
    $I_k^{(l+1/2)} \leftarrow$ Source iteration on $\eta\chi_k \sum_{g=1}^{N_g} \sigma_{f,g} I_g^{n+1}$, using (18a)
    $r^{(l+1/2)} \leftarrow \sum_{g=1}^{N_g} \sigma f, g \left(I_g^{l+1/2} - I_g^l\right)$
    $\mathcal{E}^l \leftarrow$ Grey correction equation using (18b)
    $I_k^{(l)} \leftarrow I_k^{(l+1/2)} + \mathcal{E}^{l+1/2} \frac{\langle\sigma_t\rangle\chi_k}{\sigma_{t,k}}$ using (18c)
  end while
  $I_k^n \leftarrow I_k^{(l+1)}$
  Recover $\Delta T^{n+1/2}$ from I_k^n
  if $K \neq 0 \wedge u \neq 0$ **then**
    Solve thermal diffusion equation for ΔT^{n+1}
  end if
  $T^{n+1} \leftarrow T^n + \Delta T^{n+1/2} + \Delta T^{n+1}$
end while

2. Numerical Implementation

Using the multigroup diffusion equation with Backward Euler, lagged coefficients, and implicit temperature to approximate a 1D thermal radiative transfer (TRT) problem gives, at each time step, Equation 11 can be split into two first-order ODEs,

$$\frac{\partial F_k^{n+1}}{\partial x} + \left(\sigma_a + \sigma_{f,k}\right) I_k^{l+1} = \eta\chi_k \sum_{g=1}^{N_g} \sigma_{f,g} I_g^l + S_k \tag{19a}$$

$$F_k^{n+1} = -D_k \frac{\partial I_k^{n+1}}{\partial x} \tag{19b}$$

$$0 \leq x \leq X, \qquad k = 1, \ldots, N_g$$

where temperature is recovered using Equation 7. Boundary conditions for these equations are given by

$$I^+(0) = I_{in}^+ \tag{20}$$

$$I^-(X) = I_{in}^- \tag{21}$$

$$F^+(0) = F_{in}^+ \tag{22}$$

$$F^-(X) = F_{in}^- \tag{23}$$

where

$$I^{\pm} = \pm \int_0^{\pm 1} \mathcal{I}(\mu) d\mu \tag{24}$$

$$F^{\pm} = \pm \int_0^{\pm 1} \mu \mathcal{I}(\mu) d\mu \tag{25}$$

First, the spatial domain is divided into N_x cells $K_i = [x_{i-1/2}, x_{i+1/2}]; i = 1, \ldots, N_x$, where $x_{1/2} = 0$ and $x_{N_x+1/2} = X$, and in each cell K_i the width is $\Delta x_i = x_{i+1/2} - x_{i-1/2}$. Then, if temperature (and temperature-dependent coefficients) are taken to be constant within each cell, then, in the accelerated correction equation, the term $\langle D' \rangle$ becomes zero, and the grey correction equation has the same form as Equation 19, so they may be solved in the same way.

2.1 LD Formulation

First, I and F (and the source S_k) from Equation 19 are assumed to take the form

$$I(x) = I_{L,i} b_{L,i} + I_{R,i} b_{R,i} \tag{26a}$$

$$F(x) = F_{L,i} b_{L,i} + F_{R,i} b_{R,i} \tag{26b}$$

$$S(x) = S_{L,i} b_{L,i} + S_{R,i} b_{R,i} \tag{26c}$$

$$b_{L,i} = \frac{x_{i+1/2} - x}{\Delta x_i}, \qquad b_{R,i} = \frac{x - x_{i-1/2}}{\Delta x_i} \tag{26d}$$

where the multigroup index k and the time index n have been omitted without loss of generality. The weak form of Equation 19 is found by multiplying by the test functions $b_{L,i}$ and $b_{R,i}$ and integrating over the cell.

$$\int_{x_{i-1/2}}^{x_{i+1/2}} \frac{dF^{l+1}}{dx} b_{c,i} dx + (\sigma_a + \sigma_f) \int_{x_{i-1/2}}^{x_{i+1/2}} I^{l+1} b_{c,i} dx = \eta\chi \sum_{g=1}^{N_g} \sigma_{f,g} \int_{x_{i-1/2}}^{x_{i+1/2}} I_g^{(l)} b_{c,i} dx + \int_{x_{i-1/2}}^{x_{i+1/2}} S b_{c,i} dx \tag{27a}$$

$$\int_{x_{i-1/2}}^{x_{i+1/2}} F^{l+1} b_{c,i} dx = -D \int_{x_{i-1/2}}^{x_{i+1/2}} \frac{dI^{l+1}}{dx} b_{c,i} dx \tag{27b}$$

$$c = L, R; \qquad i = 1, \ldots, N_x$$

or, rewriting the $\frac{d}{dx}$ terms using integration by parts,

$$\left[F^{l+1}b_{c,i}\right]_{x-1/2}^{x+1/2} - \int_{x_{i-1/2}}^{x_{i+1/2}} \frac{db_{c,i}}{dx} F^{l+1} dx + (\sigma_a + \sigma_f) \int_{x_{i-1/2}}^{x_{i+1/2}} I^{l+1} b_{c,i} dx =$$

$$\eta\chi \sum_{g=1}^{N_g} \sigma_{f,g} \int_{x_{i-1/2}}^{x_{i+1/2}} I_g^{(l)} b_{c,i} dx + \int_{x_{i-1/2}}^{x_{i+1/2}} S b_{c,i} dx \quad (28a)$$

$$\int_{x_{i-1/2}}^{x_{i+1/2}} F^{l+1} b_{c,i} dx = -D \left(\left[I^{l+1} b_{c,i}\right]_{x_{i-1/2}}^{x_{i+1/2}} - \int_{x_{i-1/2}}^{x_{i+1/2}} \frac{db_{c,i}}{dx} I^{l+1} dx \right) \quad (28b)$$

$$c = L, R; \qquad i = 1, \ldots, N_x.$$

Using the assumed forms in Equation 26 and the interface conditions

$$I^b(x_{i+1/2}) = I^+ + I^- = \frac{1}{2}\left(I_{R,i} + I_{L,i+1}\right) + \frac{3}{4}\left(F_{R,i} - F_{L,i+1}\right) \quad (29)$$

$$F^b(x_{i+1/2}) = F^+ + F^- = \frac{1}{4}\left(I_{R,i} - I_{L,i+1}\right) + \frac{1}{2}\left(F_{R,i} + F_{L,i+1}\right) \quad (30)$$

the local equations on the interior become,

$$\frac{1}{4}\left(I_{L,i}^{(l+1)} - I_{R,i-1}^{(l+1)}\right) + \frac{1}{2}\left(F_{R,i}^{(l+1)} - F_{R,i-1}^{(l+1)}\right) + \frac{\sigma_t \Delta x_i}{6}\left(m_{LL} I_{L,i}^{(l+1)} + m_{LR} I_{R,i}^{(l+1)}\right) =$$

$$\eta\chi \sum_{g=1}^{N_g} \frac{\sigma_{f,g}\Delta x_i}{6}\left(m_{LL} I_{g,L,i}^{(l)} + m_{LR} I_{g,R,i}^{(l)}\right) + \frac{\Delta x_i}{6}\left(m_{LL} S_{L,i} + m_{LR} S_{R,i}\right) \quad (31)$$

$$\frac{1}{4}\left(I_{R,i}^{(l+1)} - I_{L,i+1}^{(l+1)}\right) + \frac{1}{2}\left(F_{L,i+1}^{(l+1)} - F_{L,i}^{(l+1)}\right) + \frac{\sigma_t \Delta x_i}{6}\left(m_{RL} I_{g,L,i}^{(l+1)} + m_{RR} I_{g,R,i}^{(l+1)}\right) =$$

$$\eta\chi \sum_{g=1}^{N_g} \frac{\sigma_{f,g}\Delta x_i}{6}\left(m_{RL} I_{L,i}^{(l)} + m_{RR} I_{R,i}^{(l)}\right) + \frac{\Delta x_i}{6}\left(m_{RL} S_{L,i} + m_{RR} S_{R,i}\right) \quad (32)$$

$$\frac{\Delta x_i}{6}\left(m_{LL} F_{L,i} + m_{LR} F_{R,i}\right) + D\left[\frac{3}{4}\left(F_{L,i} - F_{R,i-1}\right) + \frac{1}{2}\left(I_{R,i} - I_{R,i-1}\right)\right] = 0 \quad (33)$$

$$\frac{\Delta x_i}{6}\left(m_{RL} F_{L,i} + m_{RR} F_{R,i}\right) + D\left[\frac{3}{4}\left(F_{R,i} - F_{L,i+1}\right) + \frac{1}{2}\left(I_{L,i+1} - I_{L,i}\right)\right] = 0 \quad (34)$$

and on the boundaries,

$$\frac{1}{4}\left(I_{L,i}^{(l+1)}\right) + \frac{1}{2}\left(F_{R,i}^{(l+1)}\right) + \frac{\sigma_t \Delta x_i}{6}\left(m_{LL} I_{L,i}^{(l+1)} + m_{LR} I_{R,i}^{(l+1)}\right) =$$

$$\eta\chi \sum_{g=1}^{N_g} \frac{\sigma_{f,g}\Delta x_i}{6}\left(m_{LL} I_{g,L,i}^{(l)} + m_{LR} I_{g,R,i}^{(l)}\right) + \frac{\Delta x_i}{6}\left(m_{LL} S_{L,i} + m_{LR} S_{R,i}\right) + F^{in,+} \quad (35)$$

$$\frac{1}{4}\left(I_{R,i}^{(l+1)}\right)-\frac{1}{2}\left(F_{L,i}^{(l+1)}\right)+\frac{\sigma_t\Delta x_i}{6}\left(m_{RL}I_{L,i}^{(l+1)}+m_{RR}I_{R,i}^{(l+1)}\right)=$$
$$\eta\chi\sum_{g=1}^{N_g}\frac{\sigma_{f,g}\Delta x_i}{6}\left(m_{RL}I_{g,L,i}^{(l)}+m_{RR}I_{g,R,i}^{(l)}\right)+\frac{\Delta x_i}{6}\left(m_{RL}S_{L,i}+m_{RR}S_{R,i}\right)-F^{in,-} \quad (36)$$

$$\frac{\Delta x_i}{6}\left(m_{LL}F_{L,i}+m_{LR}F_{R,i}\right)+D\left[\frac{3}{4}\left(F_{L,i}\right)+\frac{1}{2}\left(I_{R,i}\right)\right]=DI^{in,+} \quad (37)$$

$$\frac{\Delta x_i}{6}\left(m_{RL}F_{L,i}+m_{RR}F_{R,i}\right)+D\left[\frac{3}{4}\left(F_{R,i}\right)-\frac{1}{2}\left(I_{L,i}\right)\right]=-DI^{in,-}. \quad (38)$$

Written in matrix form, these equations become, on the interior,

$$\begin{bmatrix}\mathbf{B}_1+\sigma\Delta x\overline{\mathbf{M}} & \mathbf{B}_2\\ D\mathbf{B}_2 & \Delta x\overline{\mathbf{M}}+3D\mathbf{B}_1\end{bmatrix}\begin{bmatrix}\vec{I}\\ \vec{F}\end{bmatrix}=\begin{bmatrix}\mathbf{M}\vec{Q}\\ 0\end{bmatrix} \quad (39a)$$

where

$$\mathbf{B}_1=\begin{bmatrix}-1/4 & 1/4 & 0 & 0\\ 0 & 0 & 1/4 & -1/4\end{bmatrix} \quad (39b)$$

$$\mathbf{B}_2=\begin{bmatrix}-1/2 & 0 & 1/2 & 0\\ 0 & -1/2 & 0 & 1/2\end{bmatrix} \quad (39c)$$

$$\mathbf{M}=\frac{1}{6}\begin{bmatrix}m_{LL} & m_{LR}\\ m_{RL} & m_{RR}\end{bmatrix} \quad (39d)$$

$$\overline{\mathbf{M}}=\frac{1}{6}\begin{bmatrix}0 & m_{LL} & m_{LR} & 0\\ 0 & m_{RL} & m_{RR} & 0\end{bmatrix} \quad (39e)$$

$$\vec{I}=\begin{pmatrix}I_{R,i-1}\\ I_{L,i}\\ I_{R,i}\\ I_{L,i+1}\end{pmatrix},\quad \vec{F}=\begin{pmatrix}F_{R,i-1}\\ F_{L,i}\\ F_{R,i}\\ F_{L,i+1}\end{pmatrix},\quad \vec{Q}=\begin{pmatrix}\eta\chi\sum_g\sigma_{f,g}I_{g,L,i}+S_{L,i}\\ \eta\chi\sum_g\sigma_{f,g}I_{g,R,i}+S_{R,i}.\end{pmatrix} \quad (39f)$$

On the left and right boundaries, the left- or right-most column of each local matrix is truncated, with the boundary conditions added to the source:

$$\begin{bmatrix}\mathbf{B}_1^c+\sigma\Delta x\overline{\mathbf{M}}^c & \mathbf{B}_2^c\\ D\mathbf{B}_2^c & \Delta x\overline{\mathbf{M}}^c+3D\mathbf{B}_1^c\end{bmatrix}\begin{bmatrix}\vec{I}^c\\ \vec{F}^c\end{bmatrix}=\begin{bmatrix}\mathbf{M}\vec{Q}\\ 0\end{bmatrix}+\begin{bmatrix}\vec{b}_F\\ D\vec{b}_I\end{bmatrix},c=L,R \quad (40)$$

where, for example,

$$\mathbf{B}_1=\begin{bmatrix}\cancel{-1/4} & 1/4 & 0 & 0\\ \emptyset & 0 & 1/4 & -1/4\end{bmatrix}=\mathbf{B}_1=\begin{bmatrix}1/4 & 0 & 0\\ 0 & 1/4 & -1/4\end{bmatrix},\text{etc}, \quad (41)$$

and the boundary vectors are

$$\vec{b}_F = \begin{bmatrix} \delta_{c,L} F^{in,+} \\ -\delta_{c,R} F^{in,-} \end{bmatrix}, \quad \vec{b}_I = \begin{pmatrix} \delta_{c,L} I^{in,+} \\ -\delta_{c,R} I^{in,-} \end{pmatrix}. \tag{42}$$

The global matrix is then a $(4N_x \times 4N_x)$ square matrix. In the Python implementation, the first $2N_x$ rows of the global matrix are composed of the first two rows of the local matrix (the zeroth moment equation), and the last $2N_x$ rows of the global matrix are composed of the last two rows of the local matrix (Fick's Law), so that the global system is of the form

$$\mathbf{G} \begin{bmatrix} I \\ F \end{bmatrix} = \begin{bmatrix} Q + b_F \\ b_I \end{bmatrix}. \tag{43}$$

In the Python code, the global matrix assembly from local matrices is performed by the function `method.assemble_global_matrix()`, and the matrix definition can be found in `tools`. For example, the interior assembly:

```
    # interior elements
    for i in range(1, nx-1):
        R_prev = (2*i)-1
        L_next = (2*i)+2
        shift  = (2*nx)

        # zeroth moment, intensity
        global_matrix[2*i:(2*i + 2),
                R_prev:L_next+1] += B_1 + (sigma[i]*dx*M_wide)
        # zeroth moment, flux
        global_matrix[2*i:2*i + 2,
                       R_prev+shift:L_next+shift+1] += B_2

        # first moment, intensity
        global_matrix[2*i +shift:2*i + 2+shift,
                      R_prev:L_next+1] += D[i] * (B_2)
        # first moment, flux
        global_matrix[2*i+shift:2*i+2+shift,
                      R_prev+shift:L_next+1+shift]+= (dx*M_wide
                                                      + D[i]*3*B_1)
```

and the boundary element assembly:

```
    # Left boundary
    # zeroth, intensity
    global_matrix[0:2, 0:3] += B_1[:, 1:] + (sigma[0]*dx*M_wide[:, 1:])
    # zeroth, flux
    global_matrix[0:2, shift:3+shift] +=  B_2[:, 1:]
    # first, intensity
    global_matrix[shift:2+shift,
```

```
                0:3] += D[0] * (B_2[:, 1:])
    # first, flux
    global_matrix[shift:2+shift,
                shift:3+shift] += ((dx*M_wide[:, 1:]) +
                                        (D[0]*3*B_1[:, 1:]))
    # Right boundary
    # zeroth, intensity
    global_matrix[shift-2:shift,
                shift-3:shift] += (B_1[:, 0:-1] +
                                (sigma[-1]*dx*M_wide[:, 0:-1]))
    # zeroth, flux
    global_matrix[shift-2:shift,
                -3:] +=  B_2[:, 0:-1]
    # first, intensity
    global_matrix[-2:,
                shift-3:shift] += D[-1] * (B_2[:, 0:-1] )
    # first, flux
    global_matrix[-2:,
                -3:] += ((dx*M_wide[:, 0:-1]) +
                                    (D[-1]*3*B_1[:, 0:-1]))
```

where the local matrices are

```
B_1 = numpy.array([[-0.25, 0.25, 0,     0   ],
                   [ 0,    0,    0.25, -0.25]])
B_2 = numpy.array([[-0.5, 0,   0.5, 0  ],
                   [ 0,  -0.5, 0,   0.5]])

M    = (1/6)* numpy.array([[2,1],
                           [1,2]])
M_wide = (1/6)*numpy.array([[0, 2, 1, 0],
                            [0, 1, 2, 0]])
```

2.2 Unaccelerated Iteration

The unaccelerated iteration is performed according to Algorithm 1. The iteration occurs in two layers, time advancement and power iterations within each time step.

- Calculate $\varkappa$ and assign all multigroup coefficients using `coeff.assign()`
- Call `unaccelerated_loop()` to solve for intensity
- Recover temperature change from intensity using `update_temperature()`

and, within `unaccelerated_loop()`:

- Begin with previous solution as initial guess
- Until $\max_k \left\| I_k^{(l+1)} \right\|_2 / \left\| I^{(l)} \right\|_2 < \epsilon$ `and` $\max_i |I_i^{(l+1)}|/|I_i^{(l)}| < \epsilon$, do for each group:

- Call `assemble_HO()` to assemble high-order (multigroup) matrix and source using previously computed coefficients, and previous iterate for the fission source
- Solve system. `scipy.sparse.linalg.spsolve()`, which uses LU decomposition, was found to perform the best

The function `assemble_HO()` calls `assemble_global_matrix()` to assemble the global matrix as described above, and calls `get_HO_source()` to compute the source as a function of the previous iterate and apply the boundary conditions.

The number of iterations is output with the solution once convergence is reached. The number of linear solves is equal to `iters * mesh.ng`.

2.3 Accelerated Iteration

The accelerated iteration is performed according to Algorithm 2. The numerical implementation is much the same as the unaccelerated case, plus the computation of grey coefficients, and another linear solve of the grey system at each step. The behavior of `solve_diffusion()` is the same at each time step in the accelerated case, but instead calls the accelerated iteration scheme:

- Calculate $\varkappa$ and assign all multigroup coefficients using `coeff.assign()`
- Call `accelerated_loop()` to solve for intensity
- Recover temperature change from intensity using `update_temperature()`

Each call of `accelerated_loop()` does the following:

- Begin with previous solution as initial guess
- Until $\max_k \left\| I_k^{(l+1)} \right\|_2 / \left\| I^{(l)} \right\|_2 < \epsilon$ `and` $\max_i |I_i^{(l+1)}|/|I_i^{(l)}| < \epsilon$, do:
  - For each group:
    - Call `assemble_HO()` to assemble high-order (multigroup) matrix and source using previously computed coefficients, and previous iterate for the fission source
    - Solve system using `scipy.sparse.linalg.spsolve()`
  - Calculate grey coefficients using `grey_constants.assign()`
  - Call `assemble_LO()` to assemble low-order (grey) matrix and source ($\eta r^{l+1/2}$).
  - Solve system using `scipy.sparse.linalg.spsolve()`

The number of linear solves in this case is equal to `iters * (1 + mesh.ng)`, since each iteration is composed of one multigroup iteration (one solve for each group) and one grey solve.

The full directory for the Python code can be found in Appendix A.

3. Tests

3.1 Fleck and Cummings Test

The Fleck and Cummings Test [2] is a slab with $X = 4$ cm, and opacity defined by

$$\varkappa_\nu(T) = \frac{\varkappa_0}{(h\nu)^3}\left(1 - e^{-\frac{h\nu}{kT}}\right) \tag{44}$$

where $h\nu$ and kT are in keV. At the left boundary,

$$\mathcal{I}_\nu|_{x=0,\mu>0} = B_\nu(T_b) \tag{45}$$

where $T_b = 1$ keV. The right boundary is vacuum

$$\mathcal{I}_\nu|_{x=x,\mu<0} = 0 \tag{46}$$

and the initial distribution in the domain at $t = 0$ is given by

$$\mathcal{I}_\nu(t = 0) = B_\nu(T_0) \tag{47}$$

where $T_0 = 1$ eV. The material heat capacity is given by

$$C_v = 0.5917 a_R (T_b)^3. \tag{48}$$

For the diffusion problem, these conditions become:

$$I^{in,+} = \int_0^1 \mathcal{I}(\mu)|_{x=0} d\mu = \frac{1}{2}\beta_k(T_b) \tag{49}$$

$$I^{in,-} = -\int_0^{-1} \mathcal{I}(\mu)|_{x=0} d\mu = \frac{1}{2}\beta_k(T_b) \tag{50}$$

$$F^{in,-} = \int_0^{-1} \mu\mathcal{I}(\mu)|_{x=0} d\mu = \frac{1}{4}\beta_k(T_b) \tag{51}$$

$$F^{in,-} = -\int_0^{-1} \mu\mathcal{I}(\mu)|_{x=0} d\mu = -\frac{1}{4}\beta_k(T_b) \tag{52}$$

3.2 Test A1, steady-state solution

The opacity and initial condition for this test are the same as for the Fleck and Cummings test, but with different boundary conditions. The initial radiation and temperature distribution remain

$$T(x)|_{t=0} = T_0 \tag{53}$$
$$\mathcal{I}_\nu|_{t=0} = B_\nu(T_0) \tag{54}$$

and the boundary conditions are

$$\mathcal{I}_\nu|_{x=0,\mu>0} = B_\nu(T_0) \tag{55}$$
$$\mathcal{I}_\nu|_{x=X,\mu<0} = B_\nu(T_0). \tag{56}$$

Then, for the diffusion method, this becomes

$$I^{in,+} = \frac{1}{2}\beta_k(T_0) \tag{57}$$
$$I^{in,-} = \frac{1}{2}\beta_k(T_0) \tag{58}$$
$$F^{in,+} = \frac{1}{4}\beta_k(T_0) \tag{59}$$
$$F^{in,-} = -\frac{1}{4}\beta_k(T_0) \tag{60}$$
$$I(x)|_{t=0} = \beta_k(T_0) \tag{61}$$
$$F(x)|_{t=0} = 0. \tag{62}$$

3.3 Test A2, one group

3.4 Fourier-informed tests

A. Python Functions

- /
  - method ..... All iteration, acceleration
    - assemble_global_matrix() ..... Assemble global LD FEM matrix
    - get_HO_source() ..... for group k, compute fission source, time differencing source, external source, and add boundary conditions
    - get_LO_source()
    - assemble_HO() ..... Compute global matrix and source, assemble into class
    - assemble_LO()
    - unaccelerated_loop() ..... For a single time step. Call assemble_HO and solve system until convergence is reached
    - accelerated_loop() ..... Same as unaccelerated_loop(), but includes grey correction term. Assemble and solve HO system, compute grey coefficients, solve grey system, get corrected iterate, until convergence
    - update_temperature() ..... Equation 7
    - solve_diffusion() ..... handles time advancement. At each time step, call appropriate ...loop(), record results
  - tools ..... Data classes, plotting tools, local matrices
    - B_1, B_2, M, M_wide ..... Local LD matrices
    - dbl() ..... Repeat mesh.nx array to be 2*mesh.nx. Prepares piecewise constant values (opacity, temperature, etc) for use in LD solution
    - Transport_solution ..... Contains intensity and flux

- Discretization ..... Contains discretization parameters dt, dx, group structure, etc, and constants for computational units
- Global_system ..... Global matrix and source vector
- MG_coefficients ..... $\sigma_k, \eta, \chi_k, \frac{\partial \beta}{\partial T}, \beta_k, D_k, S_k, \varkappa_k$
- Grey_coeff ..... $\langle D \rangle, \langle \sigma_f \rangle, \langle \sigma_t \rangle, r$
- LD_plottable ..... Prepares LD solution for plotting
- plot_LD_groups()
- plot_LD_grey()

physics ..... Physical constants, Planck Function and derivative

- cumulative_sigma() ..... Group Stefan-Boltzmann constant
- group_planck() ..... Grouped Planck spectrum
- cumulative_dsigma_dt()
- group_db_dt()

References

[1] J. E. Morel, E. W. Larsen, and M. Matzen, "A synthetic acceleration scheme for radiative diffusion calculations," *Journal of Quantitative Spectroscopy and Radiative Transfer*, vol. 34, no. 3, pp. 243–261, 1985.

[2] J. A. Fleck Jr and J. Cummings Jr, "An implicit monte carlo scheme for calculating time and frequency dependent nonlinear radiation transport," *Journal of Computational Physics*, vol. 8, no. 3, pp. 313–342, 1971.